Can tomorrow change yesterday's decision?

Date: 24 October 2026. Task L002d: selected future-price invariance checks. Sidiq review status: Not yet reviewed. Every input is synthetic; no market sample is used.

Research question and hypothesis

If the information available at a decision stays fixed, can changing later fill evidence change that decision? We predict that the existing C++ delayed order preserves its requested quantity and decision cash/shares across valid alternative futures. Later balances and fill status may differ. We also predict that a deliberately leaking implementation fails this test.

L002a/b separated observation, decision and execution. L002c implemented that separation on October 23 and was integrated today after checking its provenance. Today's one new concept is invariance: which recorded fields must remain the same when only later information changes? No new C++ production syntax or strategy family is introduced.

Protocol, set before implementation: hold the observation/account prefix fixed; vary valid future prices, missing fill evidence and later resolution times; compare actual C++ decision projections across futures, then compare decisions and complete accounting with independent expectations. Require a leaking control to fail. Use fresh sanitizer builds and exact replays. Metrics are exact comparisons and detected wrong controls, not returns or trading performance.

Prerequisites and terminology

A prefix is the information fixed up to the decision: cash, shares, observed price and its availability time. A suffix is what happens afterwards: a fill price or its absence, and the resolution time. An invariant is a property that must survive the chosen change. A projection selects only the fields relevant to that property. Here it selects requested quantity, decision cash and decision shares. We do not demand equal final balances across different fills.

Observation arrives at event 0; the decision is at event 1; resolution is at event 2 or 9. These are ordered integers, not exchange timestamps. The toy rule requests two shares when observed price is at most 50, otherwise zero. A request is not a confirmed purchase.

A hand calculation before testing

Start with cash 800 and four shares. Observe 50 at event 0. Request two shares at event 1; leave cash 800 and shares four at that instant. Assume a fee of three only on accepted fills. Each row below is a separate possible future of that same prefix, not consecutive trades.

Later evidence	Request at event 1	Final cash / shares	Status
Fill at 49	2	699 / 6	filled
Fill at 52	2	693 / 6	filled
Fill at 399	2	800 / 4	rejected_by_model
No fill evidence	2	800 / 4	unresolved

For fill 49, debit is 2*49+3=101; cash becomes 699. For fill 52, debit is 107; cash becomes 693. For fill 399, debit would be 801, exceeding cash 800. The model refuses that buy without charging a fee. It must not retroactively erase the earlier two-share request.

Mathematics, units and the test boundary

Let P be one valid prefix and F be a valid future suffix. Let T(P,F) be the returned C++ trace. Define D(T) = (request, decision_cash, decision_shares). The tested relation is: D(T(P,F1)) = D(T(P,F2)) for each selected pair of valid suffixes sharing P.

The separate decision anchor is D = (n,C,q), where n=2 if observed_price <= 50 and n=0 otherwise. C is whole money units; q and n are whole shares; price p is money per share; fee f is whole money units. An accepted fill gives C_final = C-n*p-f and q_final=q+n. Rejection or missingness leaves C and q unchanged, with zero charged fee.

A fixed no-order implementation would satisfy invariance by doing nothing every time. That is why the independent decision anchor is also necessary. Invariance checks sensitivity to the future; it does not establish that the original decision rule is correct.

Only valid complete traces enter today's relation. A malformed future price such as zero invalidates the existing wrapper before it returns a valid trace. We do not compare that invalid record as if it represented a valid past decision. L002c's fifteen invalid-trace checks remain separate. This distinction limits the claim; a persistent event engine would need its own immutable decision record and lifecycle tests.

Read the C++ boundary slowly

The existing call is `decide_quantity(observed_price, available_event, decision_event)`. The arguments name information available before the request. The helper has no fill-price argument. Its return value is stored in trace.request. The initialized at_decision snapshot holds the original cash and shares; later try_buy receives only after_resolution by reference. The complete wrapper does receive future evidence, so testing its recorded past fields still matters.

The harness compiles the unchanged validated_buy.cpp and delayed_order.cpp with a temporary driver. It prints both snapshots, status, fee and the caller's original account for each case. Python groups the printed rows by their fixed prefix and compares their three past fields. It then checks the rule anchor, unchanged caller, and Decimal per-share resolution reference. The C++ source stays byte-identical to the integrated L002c fragment.

Exact selected grid and results

Five observed prices (1,49,50,51,10000) and four cash/share pairs (800/4,107/4,800/9999,0/4) give 20 prefixes. Nine fill choices (1,49,50,51,52,398,399,10000,absent) and two resolution events (2,9) give 18 suffixes per prefix: 360 traces. Prices 398 and 399 straddle the 800-cash affordability boundary: debits 799 and 801. The position-cap and zero-cash prefixes retain requests.

All 340 within-prefix comparisons pass: 17 comparisons to the first suffix for each prefix. All 360 complete outcomes match the Decimal reference; six hand anchors agree first. The counts are 66 filled, 126 model-rejected, 24 unresolved and 144 no-order. Each request and snapshot remains fixed even when its eventual fill status changes. No coverage percentage is claimed.

Deliberate failures and reproducibility

Four temporary C++ source mutations compile warning-clean, run twice, and are detected. They never replace production source. The first uses a future fill price in the signal; the second suppresses the request when a future fill will be unaffordable; the third copies final state over the past snapshot. All three fail the actual paired projection comparison. The fourth always requests nothing; it passes that relation but fails the decision anchor. These controls show that the checks discriminate these specific wrong implementations.

Run `make cpp-future-invariance` from the repository root. The normal command runs the correct build and all four controls, with two executions per build. Two standalone invocations produce byte-identical result JSON. An actual Python -O invocation is refused, since disabled assertions would invalidate these verification tools. The driver compiles with C++17, -Wall, -Wextra, -Wpedantic, -Werror and undefined-behavior instrumentation with no recovery.

Results and the selected fixture are in experiments/portfolio_accounting/future_invariance_results.json and future_invariance_cases.json. The result pins source/fixture/output hashes and saves the six hand traces, status counts and detected controls. The verifier is scripts/verify_future_invariance.py. The Make target is included in make check; full quality, Python and existing C++ checks pass. A separate validation record describes the exact commands, PDF checks and publication boundary.

Costs, failures and limitations

The fixed fee of three and whole-unit prices are invented assumptions. No spread, market impact, borrowing, shorting, partial fills, taxes or settlement is modelled. A valid fill price is supplied by the fixture; a test does not show whether such execution was attainable. The affordability guard is this model's rule, not a description of exchange rejection.

This is a selected grid, not exhaustive coverage of all integer prefixes, histories or event orders. Event availability/decision indices stay 0/1. We perturb valid suffixes, including missingness and two later times, but not data revisions, multiple orders, retries or persistent state. The helper's signature and these checks support a bounded causal separation claim; they are not a proof that a general backtester has no look-ahead bias.

UBSan checks selected runtime errors on executed paths. It does not understand financial causality. Likewise, matching a Decimal reference alone cannot replace the paired check: shared future-information mistakes could otherwise agree in both implementations. No empirical profitability, annualised return or personal mastery is established.

Interview explanation and next lesson

I fix the information available at decision time, then change only later evidence. Earlier requests and saved account snapshots must match; later accounting is allowed to differ. I combine this relation with hand anchors, a separate accounting reference and a leaking control, because any one test alone can miss a wrong implementation.

Next is L002e: explain one adverse-fill cost scenario slowly. L002f reproduction follows. C003b/C004/C005/Q011-13 and Q004a/Q005b remain gates. Trend and later windows stay conditional. Today's draw of two funds the pending L002c integration and this independent validation lesson. Personal review stays Not yet reviewed. No input blocks synthetic continuation; lawful source access is needed before real data acquisition. Stop new research after December 31.

Primary sources consulted October 24

QuantConnect, Timeslices: its time frontier exposes data according to simulated time and illustrates why future information must be withheld. Our event-index convention and paired test design are teaching choices, not a claim to reproduce LEAN's implementation.
https://www.quantconnect.com/docs/v2/writing-algorithms/key-concepts/time-modeling/timeslices

SEC Investor.gov, Types of Orders: the last-traded price need not equal execution price. This motivates separating observed price from assumed future fill. It does not validate our synthetic fills, fee or absence-of-fill convention.
https://www.investor.gov/introduction-investing/investing-basics/how-stock-markets-work/types-orders

Clang, UndefinedBehaviorSanitizer: selected undefined behavior is checked at runtime; -fno-sanitize-recover=all terminates on a detected error. This complements the logical tests.
https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

Check your understanding

1. Fill price changes from 49 to 52 after the same observation of 50. Which fields must stay fixed, and which final balance changes? Calculate the cash difference.

2. Why should a later unaffordable fill not change the earlier request from two to zero?

3. A faulty implementation always requests zero. Does it pass invariance? What catches it?

4. Why is changing a future fill to zero excluded from today's paired relation?

5. What do 360 passing traces and four detected mutations fail to prove?

Suggested answers

1. Request remains two and decision cash/shares remain 800/four. Final cash changes from 699 to 693, a difference of -6 money units: two shares times a three-unit price increase. Final shares remain six and fee remains three in both accepted fills. Different final cash is expected; requiring equality would test the wrong property.

2. The price at the later resolution was not available at the decision. With observed price 50, the declared rule already requested two. A fill at 399 would cost 801, so the 800-cash model rejects at resolution, preserving cash and shares and charging no fee. A request and its eventual acceptance are distinct facts. This rule does not reserve cash at decision time.

3. Yes: constant zero is unchanged under every future. The hand/rule anchor expects two for observed prices at most 50, so it rejects that implementation. A useful relation must be paired with evidence that at least the intended baseline behavior is present.

4. A supplied zero is outside the valid price domain. The complete wrapper invalidates the record before returning a valid trace. Today's relation compares only valid complete traces. Missing evidence is different: absent fill is valid and leaves an outstanding request unresolved. The existing invalid-trace suite checks malformed evidence separately.

5. They do not prove all histories are causal, all possible mutations are detected, a real fill is achievable, the strategy earns money, or Sidiq has independently reproduced it. They establish exact behavior on the declared grid and sensitivity to four specified defects.